Part 1 — MCP and the Fetch server

Amir Darwich · EECE 503P/798S Agentic Systems · C5 Agent Tools

What the article says

Ksenia Se's article explains why the Model Context Protocol, open-sourced by Anthropic in November 2024, took off in early 2025. Before MCP, every model-to-tool link was custom glue, so M applications and N tools meant M×N integrations. MCP is an open, model-agnostic standard: each side implements it once, giving M+N. The article contrasts it with ChatGPT plugins (proprietary, mostly one-way lookups), agent frameworks such as LangChain (which standardise tools for developers, whereas MCP lets the model discover them at runtime), and RAG (passive retrieval, while MCP can also act). It is candid about gaps: weak tool descriptions still cause poor tool choice, and remote deployment, authentication and monitoring were unfinished.

The three roles fit together as follows. The **Host** is the application the user talks to (Claude Desktop, an IDE, or my Agents SDK program); it owns the model and decides what the model may do. Inside it, one **Client** per server keeps a 1:1 session, discovers capabilities, and relays JSON-RPC calls. Each **Server** exposes three primitives: *tools* the model calls, *resources* the application reads, and *prompts* the user invokes.

The server I explored: Fetch

The reference Fetch server exposes one tool, `fetch(url, max_length=5000, start_index=0, raw=false)`, plus a `fetch` prompt.

What it does well. It has a well-defined interface: strictly typed arguments with bounds, accurate read-only and open-world annotations, and failures (robots.txt refusal, HTTP errors, refused private addresses) returned as `isError` results the model can read. Isolation is respected: one capability, with chunked reading through `start_index` instead of a second tool. It also blocks private and cloud-metadata addresses by default, closing the obvious SSRF hole.

Where it falls short of the class principles.

1. The tool description mixes documentation with behavioural instructions to the model ("this tool now grants you internet access…"). That is exactly what tool-poisoning attacks exploit, so a host cannot easily tell trusted guidance from manipulation.
2. `raw` is a mode flag that makes one tool behave as two (simplified markdown vs. raw HTML), the "and" smell from class.
3. Reading past the end returns an `<error>` string as normal content instead of `isError`, which breaks the otherwise consistent error contract.
4. Fetched pages are untrusted text returned without any provenance marking, so the server is a direct channel for indirect prompt injection. Defending against that is left entirely to the host.

These four points shaped my own design.

Reflection

I drew the lines between the tools by asking what each one is allowed to change and who decides. `search_rooms` (Data) can only read, so the agent may call it freely and repeatedly; it has a read-only annotation and needs no approval. `book_room` (Action) changes exactly one thing, one booking for the signed-in student, and assumes the decision has already been made: the user named the room. `plan_study_session` (Orchestration) exists for the case where the user delegates the decision ("anything between 4 and 6"), and its only job is the policy that connects the other two: scan half-hour starts, take the earliest slot with a free room, prefer the tightest fit, and retry the next room if one is taken in between. I kept the policy out of the Action tool so `book_room` stays predictable, and I made the orchestrator call the same service functions the other two tools wrap, so the three can never disagree about availability. Several things I first wanted as arguments became structural decisions instead. The booking holder comes from the transport (an environment variable the host sets), not from a model-chosen argument, so the agent cannot book in someone else's name. Every side-effecting call needs human approval in the host. The host also pins a hash of each tool's definition and refuses to start if it changes, which addresses the description-poisoning weakness I criticised in the Fetch server.

Three tools felt right for this use case, but only because each is narrow. A real assistant would need `cancel_booking` and `list_my_bookings` almost immediately; I exposed the latter's read path as a resource (`studyspot://bookings/{id}`) to stay within the brief. The hardest part was failure design: about half the test suite is predictable errors (past dates, closing time, quota, races, injected text), each with a stable code and a hint the model can act on. Next steps: add cancellation as a fourth, approval-gated Action tool; replace the demo dataset with the library's real reservation API behind the same interface; move to streamable HTTP with OAuth so identity comes from a real login; and add an LLM-based classifier alongside the pattern-based injection filter, which is a heuristic rather than a guarantee.